A Correctness Note for the Factor Library Reward Mechanism

April 15, 2026

Abstract

This note formalizes the factor-library mechanism implemented in `src/environment/factor_library.py` and `src/environment/circuit_game.py`. We prove five implementation-faithful claims. First, the factor library never changes the transition rule or the exact success condition of the circuit-construction environment; it only changes reward shaping and cross-episode bookkeeping. Second, every initial factor subgoal returned by the library is a genuine nontrivial divisor of the target polynomial modulo p. Third, every dynamically discovered residual or exact quotient subgoal is algebraically valid. Fourth, both completion bonuses are exact certificates that the current state is one operation away from the target: the additive bonus certifies one addition away, and the multiplicative bonus certifies one multiplication away. Fifth, every polynomial stored in the session library has a witnessed circuit construction from some successful episode.

1 Setup

Let p be the environment modulus, let n be the number of variables, and let d be the configured per-variable degree cap. The implementation stores each polynomial as a dense coefficient tensor of shape $(d+1)^n$, so the exact algebraic object manipulated by `FastPoly` is the truncated ring

$$R_{p,d,n} := \mathbb{F}_p[x_0, \ldots, x_{n-1}] \big/ \langle x_0^{d+1}, \ldots, x_{n-1}^{d+1} \rangle.$$

Addition is exact in this quotient ring, and multiplication is exact in this quotient ring because the code performs convolution and then discards monomials whose exponent in some variable exceeds d.

Let

$$\mathcal{B} := \{x_0, \ldots, x_{n-1}, 1\} \subset R_{p,d,n}$$

be the base nodes present at the start of every episode. A state is an ordered tuple

$$s_t = (v_0, \ldots, v_{m_t-1}) \in \mathcal{S},$$

where the first $n+1$ entries are the base nodes and each later node is formed from earlier nodes by one binary operation $+$ or $\times$.

An action is a triple $(\star, i, j)$ with $\star \in \{+, \times\}$ and $0 \le i \le j < m_t$. The next node is

$$v_{m_t} = v_i \star v_j \in R_{p,d,n}.$$

For a fixed target polynomial $T \in R_{p,d,n}$, the success predicate is

$$\mathrm{Succ}(s_t) \iff \exists k < m_t \text{ such that } v_k = T.$$

The session-level factor library is a map

$$\mathcal{L} : R_{p,d,n} \setminus \mathcal{B} \to \mathbb{N}$$

that records polynomials seen in successful episodes, together with the first step index at which they were observed in that episode. During a single episode, the environment also maintains a subgoal set $\mathcal{G}_t \subseteq R_{p,d,n}$ and a set of already-claimed subgoals to prevent double counting.

Reduction maps. Let

$$\rho_p : \mathbb{Z}[x_0, \ldots, x_{n-1}] \to \mathbb{F}_p[x_0, \ldots, x_{n-1}]$$

be coefficient-wise reduction modulo p, and let

$$\tau_d : \mathbb{F}_p[x_0, \ldots, x_{n-1}] \to R_{p,d,n}$$

be the quotient map. Their composition $\pi := \tau_d \circ \rho_p$ is exactly what the code implements when it converts a SymPy polynomial into a `FastPoly`.

2 Reward Decomposition

Ignoring action masking and termination caps, the reward implemented in `CircuitGame.step()` can be written as

$$\begin{aligned} r_t = c_{\text{step}} &+ \mathbf{1}\{\text{Succ}(s_{t+1})\}\, c_{\text{succ}} \\ &+ \mathbf{1}\{\neg\text{Succ}(s_{t+1})\}\big(\gamma\Phi(s_{t+1}) - \Phi(s_t)\big) \\ &+ c_{\text{fac}} I_t^{\text{fac}} + c_{\text{lib}} I_t^{\text{lib}} + c_{\text{comp}} I_t^{+} + c_{\text{comp}} I_t^{\times}, \end{aligned}$$

where:

- Φ is the term-similarity potential used by the existing shaping rule;
- I_t^{fac} indicates that the newly built node belongs to the current episode's active subgoal set and has not been rewarded before;
- I_t^{lib} indicates that this subgoal was also in the cross-episode library;
- I_t^{+} and $I_t^{\times}$ are the additive and multiplicative completion indicators.

The potential-based term is standard shaping in the sense of Ng–Harada–Russell (1999). The factor-library terms are different: they are not policy-invariance corrections, but explicit guidance toward reusable intermediates. The goal of this note is therefore *soundness*, not policy-invariance: we prove that these bonuses point toward algebraically valid subcomputations and do not alter the exact success criterion.

3 State-Semantics Invariants

Lemma 1 (Circuit semantics)**.** *Every node v_k created by the environment equals the polynomial computed by its corresponding arithmetic circuit in $R_{p,d,n}$.*

Proof. The claim is immediate by induction on the creation order. The base nodes are exactly $x_0, \ldots, x_{n-1}, 1$. If the claim holds for all previously created nodes and the environment applies $(\star, i, j)$, then the new node is defined to be $v_i \star v_j$ using the ring operations implemented by `FastPoly`. Those operations are the quotient-ring operations of $R_{p,d,n}$, so the semantic equation is preserved. □

Proposition 2 (The factor library does not change what counts as success)**.** *Fix an episode target T and an action sequence $a_0, \ldots, a_{K-1}$. Suppose the environment is run twice from the same initial state and with the same action sequence, once with the factor-library logic disabled and once with it enabled. Then both runs produce the same sequence of nodes and the same terminal success/failure outcome. The only differences are reward values and auxiliary bookkeeping sets.*

Proof. In `CircuitGame.step()`, node creation depends only on the decoded action, the current node list, and the deterministic `FastPoly` operations. The success predicate is the equality test $v_{\text{new}} = T$, and the done flag depends only on success, the step cap, and the node cap. The factor-library logic is executed only after the new node has been created and success has been checked. It modifies reward and updates auxiliary sets $(\mathcal{G}_t, \mathcal{L})$, but it does not rewrite the node list, the target, or the success predicate. □

4 Soundness of Initial Factor Subgoals

At reset time, the code converts the target T into a representative $\widetilde{T} \in \mathbb{Z}[x_0, \ldots, x_{n-1}]$ with coefficients in $\{0, 1, \ldots, p-1\}$ and asks SymPy for a factorization

$$\widetilde{T} = c \prod_{\ell=1}^{m} \widetilde{f}_\ell^{\,e_\ell} \qquad \text{in } \mathbb{Z}[x_0, \ldots, x_{n-1}].$$

Each candidate factor is then reduced by π, and the implementation discards scalars, base nodes, zero polynomials, duplicates, and the target itself.

Proposition 3 (Every returned initial factor is a true divisor mod p)**.** *Let $g \in R_{p,d,n}$ be any element returned by `factorize_target(T)`. Then there exists $h \in R_{p,d,n}$ such that*

$$T = gh \qquad \textit{in } R_{p,d,n}.$$

Moreover, g is nonzero, non-scalar, not a base node, and not equal to T.

Proof. By construction, $g = \pi(\widetilde{f}_i)$ for some integer factor $\widetilde{f}_i$ in the above factorization, and the filters guarantee that g is nonzero, non-scalar, not in $\mathcal{B}$, and not equal to T. Let

$$\widetilde{h} := c\, \widetilde{f}_i^{\,e_i - 1} \prod_{\ell \neq i} \widetilde{f}_\ell^{\,e_\ell} \in \mathbb{Z}[x_0, \ldots, x_{n-1}].$$

Then $\widetilde{T} = \widetilde{f}_i \widetilde{h}$ in the integer polynomial ring. Applying π gives

$$T = \pi(\widetilde{T}) = \pi(\widetilde{f}_i)\pi(\widetilde{h}) = gh$$

with $h := \pi(\widetilde{h}) \in R_{p,d,n}$. □

Remark 4. Factoring over $\mathbb{Z}$ and then reducing modulo p is therefore sound for discovering divisors: each individual factor still divides the target after reduction. The choice of factoring over $\mathbb{Z}$ is about numerical robustness and implementation convenience, not about changing the divisor relation.

5 Soundness of Dynamic Discovery

Dynamic discovery is gated by the condition that the newly built node $v := v_{\text{new}}$ is already in the session library. This gating is purely computational; it reduces SymPy calls. The algebraic claims below do not depend on why the gate was opened.

5.1 Residual subgoals

The environment computes the residual

$$R_v := T - v \in R_{p,d,n}.$$

It may add R_v itself to the active subgoal set (unless it is zero or a base node), and it may also factorize R_v to add nontrivial factors of R_v.

Proposition 5 (Residual soundness)**.** *If the environment adds the residual R_v to the active subgoal set, then*

$$v + R_v = T \qquad \text{in } R_{p,d,n}.$$

If the environment additionally adds a factor g returned by `factorize_poly(R_v)`*, then g is a nontrivial divisor of R_v in $R_{p,d,n}$.*

Proof. The identity $v + R_v = T$ is true by definition of R_v. The second claim is exactly the previous proposition applied to R_v instead of T, because `factorize_poly` uses the same factorization-and-filtering procedure on its input polynomial. □

5.2 Exact quotients

The exact-quotient routine converts T and v into integer representatives $\widetilde{T}, \widetilde{v}$, computes

$$\widetilde{T} = \widetilde{v}\widetilde{q} + \widetilde{r} \qquad \text{in } \mathbb{Z}[x_0, \ldots, x_{n-1}],$$

and accepts the quotient precisely when $\pi(\widetilde{r}) = 0$.

Proposition 6 (Exact quotient soundness)**.** *If* `exact_quotient(T,v)` *returns $q \in R_{p,d,n}$, then*

$$T = vq \qquad \text{in } R_{p,d,n}.$$

If the environment further factorizes q and adds some factor g, then g is a nontrivial divisor of q in $R_{p,d,n}$.

Proof. By the division identity over $\mathbb{Z}$,

$$\widetilde{T} = \widetilde{v}\widetilde{q} + \widetilde{r}.$$

Applying π and using the acceptance condition $\pi(\widetilde{r}) = 0$ yields

$$T = vq + 0 = vq \qquad \text{in } R_{p,d,n},$$

where $q = \pi(\widetilde{q})$. The factor statement again follows from the soundness of `factorize_poly` applied to the polynomial q. □

6 Completion Bonuses

6.1 Additive completion

The additive completion bonus fires when the residual $R_v := T - v$ is already present among the old nodes of the current state.

Theorem 7 (Additive completion is an exact one-step certificate)**.** *Suppose the additive completion bonus fires at a state whose newly created node is v. Then there exists an already existing node u such that*

$$u = T - v \qquad \textit{and hence} \qquad u + v = T$$

in $R_{p,d,n}$. Therefore the current state is exactly one addition away from the target.

Proof. By the firing condition, there is an existing node u whose canonical key matches that of $R_v = T - v$. Canonical keys are just serialized coefficient tensors, so equality of keys means equality in $R_{p,d,n}$. Hence $u = T - v$, and therefore $u + v = T$. □

6.2 Multiplicative completion

The multiplicative completion logic computes the exact quotient $Q_v := T/v$ when it exists, and then fires if either:

1. Q_v already appears as an existing node, or
2. Q_v already appears as an existing node.

Proposition 8 (Multiplicative completion is an exact one-step certificate)**.** *If the multiplicative completion bonus fires, then the exact quotient Q_v already appears as an existing node, and the current state is exactly one multiplication away from the target.*

Proof. By exact quotient soundness, $T = vQ_v$ in $R_{p,d,n}$. If Q_v is already an existing node, one multiplication of that node with v produces T. □

7 Correctness of Library Accumulation

After a successful episode, the code registers every non-base node built during that episode.

Proposition 9 (Every library entry has a witnessed circuit)**.** *Let f be any polynomial stored in the session library after some number of episodes. Then there exists a successful episode and a concrete arithmetic circuit within that episode whose output equals f. Moreover, the stored step count for f is an upper bound on the minimum number of operations needed to produce f in the environment.*

Proof. The library is updated only by `register_episode_nodes()`, which is called only after `is_success=True`. That routine iterates over the actual node list created during the successful episode and inserts each non-base node into the library. By the circuit-semantics lemma, each such node has a witnessed circuit in that episode. The recorded step number is the index at which the node appeared in that witnessed construction, so it is a valid upper bound on the optimal construction cost. The library keeps the minimum over all observed successful episodes, which preserves the upper-bound property. □

8 Main Corollary

Corollary 10 (Soundness summary)**.** *Under the exact semantics of the implemented ring $R_{p,d,n}$:*

1. *the factor library never changes the environment's transition rule or success predicate;*
2. *every initial factor subgoal is a genuine nontrivial divisor of the target;*
3. *every dynamically added residual or exact quotient subgoal is algebraically valid;*
4. *the additive completion bonus is an exact one-step certificate;*
5. *the multiplicative completion bonus is an exact one-step certificate; and*
6. *every library entry corresponds to a witnessed reusable subcircuit from a successful episode.*

9 Relation to the Ordinary Polynomial Ring

The implementation is exact in $R_{p,d,n}$, the truncated quotient ring. If one wants claims in the ordinary polynomial ring $\mathbb{F}_p[x_0, \ldots, x_{n-1}]$ instead, it is enough to assume that all intermediate polynomials appearing in the trajectories of interest have per-variable degree at most d. Under that assumption, the quotient map τ_d is injective on those trajectories, so all equalities proved above in $R_{p,d,n}$ lift to equalities in the usual polynomial ring.

10 Practical ICML Framing

For an ICML submission, a strong theorem statement is therefore:

> The factor library is a sound reward-guidance mechanism for reusable subcomputations. It does not alter which action sequences compute the target; it only adds bonuses attached to algebraically valid factors, residuals, and exact quotients, together with a session-level memory of witnessed successful intermediates.

Scope note. This note formalizes the reference factor-library implementation in `src/environment/factor_libra` and `src/environment/circuit_game.py`. The JAX PPO+MCTS runtime mirrors the same sound core rewards for initial factor hits, library bonus, additive completion, multiplicative completion, and direct residual/quotient discovery, but it does not recursively SymPy-factor every newly discovered residual/quotient inside the tree-search hot path.